

KERA: IO-Aware Mixed-Precision FPGA Accelerator for Low-Latency Sensor-Fusion Attention

Theme 3 (AI Hardware Accelerators)

Adithya Sridhar, Kade Zisko, Hyeongseok Koh, Rohaan Brar

Problem & Motivation

Attention-based fusion models are a natural fit for combining sensor streams (position, velocity, environmental data) into real-time robotics decisions. But the standard way of computing attention requires building out the entire N by N score matrix before Softmax even starts. That's a lot of memory traffic spent on an intermediate result nobody needs to keep around, especially under a hard control-loop latency budget. We're building an FPGA accelerator that avoids this. Q/K/V projections use INT4 weights packed against INT8 activations, and attention itself ($QK^T/\sqrt{d}$ then Softmax then AV) happens tile by tile so the full score matrix never gets materialized. The goal isn't just speed. We want to measure how much of the gain comes from quantization versus tiling. Starting small with one attention head, roughly 64 to 128 tokens (sensor readings or time steps), head dimension 32 to 64, and 8x8 tiles.

Technical Approach

1. *Model and quantization (Adithya)*: Train a small sensor-fusion attention layer, building on ARES path-optimization work. Quantize W_Q , W_K , and W_V to INT4 with INT8 activations, packed, with a wider accumulator before requantizing. A Python reference model provides ground truth for RTL verification.
2. *Core RTL (Kade)*: SystemVerilog projection engine computing Q/K/V from packed INT4 BRAM weights, plus the tiled datapath for scaled QK^T and AV accumulation. This includes the MAC pipeline, tile scheduling, and control logic needed to stream results without ever storing the full matrix.
3. *Sensor and FPGA interface (Rohaan)*: Simple ESP32 UART/SPI link for the live demo. Generates representative sensor data, packetizes it, and feeds it into the FPGA's input buffers. Kept intentionally basic so effort stays on the accelerator.
4. *Online Softmax and memory (Hyungseok)*: Fixed-point Softmax tracking running row maxima and normalization with LUT-approximated exp and reciprocal. BRAM buffering holds Q/K/V and partial outputs while discarding each score tile once it's used.
5. *Ablation study (team)*: Three configurations run side by side to isolate what's actually helping. A is INT8 weights and activations with materialized attention, the baseline. B is INT4 weights plus INT8 activations, still materialized. C is the full design with tiling added. A to B shows the quantization gain. B to C shows the tiling gain. If time is short, A and B can be cycle-accurate models instead of full RTL, prioritizing a finished, verified C.

Prototype/Deliverable

FPGA accelerator (config C) benchmarked against A and B. RTL and cycle simulation for latency, synthesis-based resource and power numbers for RTL-implemented configs, and a live demo running off the ESP32 sensor feed.

Evaluation

Across all three configs: cycle latency, memory traffic, peak storage, and numerical error versus the Python reference. For RTL-implemented configs, synthesis also gives DSP, LUT, FF, and BRAM usage, achievable clock speed, and estimated power. The demo video walks through the pipeline live: sensor input, projection, tiled $QK^T/\sqrt{d}$, online Softmax, and AV accumulation, alongside the three-way comparison.